

Deep Learning

Personal Notes

Mihir Rao

Contents

1	Introduction	2
1.1	Why Representations Matter	2
1.2	From Programming to Machine Learning	3
1.3	Shallow Learning and Hand-Designed Features	3
1.4	Multiple Layers of Learned Representations	4
1.5	Why Deep Learning Became Practical	4
1.6	Capacity, Inductive Bias, and Scaling	5
1.7	Why “Neural” Networks?	6

1 Introduction

Deep learning is best understood as *representation learning at scale*. Instead of asking a programmer to specify every rule or every useful feature, a model learns a sequence of internal representations from data and adapts all of them to the task at hand. This lecture motivates that view, contrasts it with classical shallow learning, and identifies the ingredients that made modern deep learning effective.

1.1 Why Representations Matter

Machine translation provides a useful opening example. A conventional system trained only to translate English into French requires paired English–French sentences. Building a separate model for every ordered language pair scales poorly, especially for rare languages for which little parallel text exists.

A multilingual model takes both a sentence and a requested output language as inputs. Training several translation directions in one system encourages the model to encode sentences in a shared internal space before decoding them into the requested language.

Definition 1.1 (Representation). A *representation* is an internal description of an input that makes information relevant to a downstream task accessible while suppressing details that do not matter for that task.

The shared representation produces two striking effects.

- *Transfer*. Data from common languages improves translation into and out of rarer languages because the same parameters and internal structure are reused.
- *Zero-shot translation*. A system trained on English–French and English–Spanish directions can sometimes translate French directly into Spanish even without paired French–Spanish examples.

The learned language code can even be varied continuously. Interpolating between nearby language codes produces outputs that transition gradually between related languages. For less compatible pairs, an intermediate code may produce text belonging cleanly to neither endpoint. Both outcomes suggest that the model has learned a continuous geometry of language rather than a table of independent translation rules.

Note: *The internal state is not literally a human thought. The analogy is useful only because the representation captures meaning in a form that can be decoded into several languages.*

Complex inputs such as text, images, and audio do not arrive with an obvious coordinate system.

The central promise of deep learning is to learn a useful coordinate system automatically from examples.

1.2 From Programming to Machine Learning

A program is a function that maps inputs to outputs. When the mapping can be stated exactly, we write its rules by hand. Object recognition, translation, and speech understanding contain too many exceptions for that approach to be practical. Machine learning replaces hand-written rules with a parameterized function

$$f(x; \theta) = y,$$

where x is the input, y is the output, and the parameter vector θ is estimated from data.

Example: For a two-dimensional classification problem, a linear rule may predict one class whenever

$$x_1\theta_1 + x_2\theta_2 + \theta_3 \leq 0.$$

Learning chooses $\theta = (\theta_1, \theta_2, \theta_3)$ so that this boundary agrees with labeled examples. The same notation $f_\theta(x)$ can describe much more complicated functions.

Choosing the function family is just as important as choosing its parameters. A linear boundary is sensible only when the input coordinates already expose the distinction we care about. Raw pixels do not: changes in viewpoint, lighting, pose, background, and position can alter many pixels without changing an object's identity.

1.3 Shallow Learning and Hand-Designed Features

Traditional machine-learning pipelines often separate representation from prediction:

$$x \xrightarrow{\phi} \phi(x) \xrightarrow{g_\theta} \hat{y}.$$

The feature extractor ϕ is designed by a person, while only the predictor g_θ is learned. This is a compromise: the programmer no longer writes the final classification rules, but still decides how the raw input should be represented.

Good features can turn a difficult problem into a simple one. Edges, corners, and local texture statistics, for example, may make images easier to separate with a linear classifier. The difficulty is that feature engineering requires substantial domain knowledge and must often be repeated for each new task.

Deep learning moves the parameters into the feature extractor as well:

$$x \xrightarrow{\phi_\alpha} \phi_\alpha(x) \xrightarrow{g_\beta} \hat{y}, \quad \theta = (\alpha, \beta).$$

Now both the representation and the predictor can improve from data.

1.4 Multiple Layers of Learned Representations

A deep model composes several simple parameterized transformations. Let $h_0 = x$ and define

$$h_\ell = f_\ell(h_{\ell-1}; \theta_\ell), \quad \ell = 1, \dots, L,$$

with output $\hat{y} = h_L$. Each layer receives the representation produced by the preceding layer. In visual recognition, early layers may respond to oriented edges, middle layers to textures or parts, and later layers to task-relevant object concepts.

Higher-level representations aim to be:

- more *abstract*, combining simple patterns into larger concepts;
- more *invariant* to nuisance changes such as pose or lighting; and
- easier for the final predictor to map to the desired output.

Definition 1.2 (Deep learning). *Deep learning* is machine learning with multiple layers of learned representations, usually implemented by a deep neural network.

When the parameters of every layer are optimized for the final objective, the procedure is called *end-to-end learning*. An image classifier, for example, can learn its early visual filters and its final decision rule from the same classification loss rather than from separate hand-written objectives.

1.5 Why Deep Learning Became Practical

The basic ideas are not new. Their impact changed as optimization methods, datasets, and computers improved.

Year	Milestone
1950	Turing discusses learning as a route to machine intelligence.
1957	Rosenblatt proposes the perceptron as a practical learning method.
1969	Minsky and Papert describe fundamental perceptron limitations.
1986	Backpropagation becomes a practical method for training deep nets.
1989	LeNet applies a neural network to handwriting recognition.
~2006	Deep neural networks regain broad research attention.
2012	AlexNet decisively improves ImageNet recognition performance.

Three resources reinforce one another:

1. *Model capacity.* Larger models with more layers can represent a broader range of functions. Vision systems progressed from the seven-layer LeNet to the eight-layer AlexNet and later to much deeper architectures such as ResNet-152.
2. *Data.* Large datasets constrain those flexible models. Image collections grew from roughly 9,000 examples in Caltech 101 and 60,000 in MNIST or CIFAR-10 to roughly 1.5 million images in the ImageNet challenge used in the lecture.
3. *Compute.* Training and evaluating these models requires enough parallel arithmetic and memory bandwidth to process the data at useful speed. Modern accelerators made model and dataset scale operationally feasible.

This dependence on scale supports two readings. Deep learning can be expensive because it consumes large models, datasets, and computation. At the same time, it is valuable precisely because many methods continue to improve when those resources increase.

1.6 Capacity, Inductive Bias, and Scaling

Definition 1.3 (Model capacity). The *capacity* of a model class is, informally, the variety of functions it can represent. A class containing only linear decision boundaries has less capacity than one containing rich nonlinear functions.

More capacity is not automatically better. A model still needs data and an effective optimization procedure to select a useful function from the larger class.

Definition 1.4 (Inductive bias). An *inductive bias* is a built-in preference that makes some learned solutions more likely than others. It is the knowledge encoded by the model class, architecture, objective, or training procedure.

Learning never eliminates inductive bias completely. The design question is how much task-specific knowledge to build in and how much to let the model acquire from data. Deep learning shifts effort away from manual feature design and toward general architectures and training machinery that can learn useful representations across tasks.

Definition 1.5 (Scaling). An algorithm *scales* when its performance can continue improving as data, model capacity, and computation increase.

The underlying themes are therefore automation, task-adapted representations, and algorithms that benefit from additional resources rather than saturating immediately.

1.7 Why “Neural” Networks?

The name comes from an early, deliberately crude analogy with biological neurons. A basic artificial unit aggregates upstream activations and applies a nonlinear activation function:

$$z = \sum_i w_i a_i + b, \quad a = \sigma(z).$$

The output activation is then transmitted to units in the next layer. Stacking many such units gives a network of learned transformations.

Real neurons have dendrites, axons, synapses, temporal dynamics, and biological mechanisms omitted by this abstraction. Artificial neural networks model only a small part of neuronal behavior, so the analogy should not be treated as a literal theory of the brain.

The resemblance nevertheless raises an interesting question: when artificial and biological systems learn similar features, is the learning algorithm biologically shared, or are similar representations simply useful solutions that any sufficiently capable learner might discover? The lecture leaves this as an open conceptual theme.

Source. These notes synthesize Lecture 1, Introduction, from UC Berkeley’s CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks, Spring 2021, taught by Sergey Levine. The multilingual translation example follows Johnson et al., Google’s Multilingual Neural Machine Translation System: Enabling Zero-Shot Translation (2016).